

**GO-CAMPUS
DYNAMIC CAMPUS SHUTTLE TRACKING AND LOAD
MONITORING SYSTEM**

THAYALAN A/L RAMASAMY

**FACULTY OF INFORMATICS AND COMPUTING
UNIVERSITI SULTAN ZAINAL ABIDIN**

2026



**GO-CAMPUS
DYNAMIC CAMPUS SHUTTLE TRACKING AND LOAD
MONITORING SYSTEM**

THAYALAN A/L RAMASAMY

**THIS THESIS IS SUMMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF COMPUTER SCIENCE (INTERNET
COMPUTING) WITH HONOURS**

**FACULTY OF INFORMATICS AND COMPUTING
UNIVERSITI SULTAN ZAINAL ABIDIN**

JANUARY 2026

DECLARATION

I hereby declare that the report is based on my original work except for quotations and citations, which have been duly acknowledged. I also declare that it has not been previously or concurrently submitted for any other degree at Universiti Sultan Zainal Abidin or other institutions.

Name: Thayalan A/L Ramasamy

Date:

ACKNOWLEDGEMENT

In the Name of Allah, the Most Gracious and the Most Merciful.

Alhamdulillah, I thank God for His grace and grace, I can prepare and complete this report successfully.

First of all, I would like to thank my supervisor, Dr Siti Sabariah Binti Abas because with guidance, the advice, and the thoughtful ideas are given g me the opportunity to prepare this report successfully.

Besides, my gratitude is also to my colleagues who share ideas, opinions, knowledge, and reminders. They helped me answer every question that was important to me in completing this report.

Thanks also to my beloved mother and father always support and motivated me to prepare for this report for Final Year Project.

I would like to take the opportunity to thank all lecturers of the Informatics and Computing Faculty for their attention, guidance, and advice in helping and sharing ideas and opinions in making this report successful.

May Allah SWT bless all the efforts that have been given in completing this report.

Thank you.

TABLE OF CONTENTS

	PAGE
DECLARATION	i
ACKNOWLEDGEMENT	ii
TABLE OF CONTENTS	iii
LIST OF ABBREVIATIONS	v
LIST OF TABLE	vii
LIST OF FIGURE	viii
ABSTRACT	ix
ABSTRAK	x
CHAPTER 1 INTRODUCTION	1
1.1 Introduction	1
1.2 Project Background	1
1.3 Problem Statement	3
1.3.1 Lack Of Real-Time Shuttle Location Information	3
1.3.2 Absence Of Shuttle Occupancy Monitoring	4
1.3.3 Uncertainty In Shuttle Arrival And Location	4
1.3.4 Inefficient Shuttle Management And Resource Allocation	4
1.4 Objectives	5
1.5 Scope	6
1.5.1 User Scope	7
1.5.2 Application Scope	7
1.6 Limitation of Work	8
1.7 Expected Result	9
1.8 Gantt Chart	10
1.9 Summary of the Chapter	10
CHAPTER 2 LITERATURE REVIEW	11
2.1 Introduction	11
2.2 Smart Transportation and IoT-Enabled Campus Mobility	11
2.3 Real-Time Vehicle Tracking and Passenger Load Monitoring	12
2.4 Cloud Computing , Dashboards and User Applications	13
2.5 Rule-Based Event-Driven Monitoring with Threshold Logic	14
2.6 Challenges and Limitations of Existed Systems	15
2.7 Summary of Literature Review	22
CHAPTER 3 METHODOLOGY	23
3.1 Introduction	23
3.2 Agile Methodology	23
3.3 Requirement Analysis	24
3.4 Software Development	25
3.5 Testing	26
3.6 Evaluation	27
3.7 System Requirement	27
3.7.1 Hardware Requirements	28
3.7.2 Software Requirements	29
3.8 System Design	30
3.9 Framework	30
3.10 Context Diagram	31

3.11 Data Flow Diagram Level 1 (DFD1)	32
3.12 Data Flow Diagram Level 2 Collect Bus Tracking Data (DFD2)	34
3.13 Data Flow Diagram Level 2 View Bus Location and Passenger Count (DFD2)	35
3.14 Data Flow Diagram Level 3 Bus Tracking Update Flow (DFD3)	37
3.15 Data Flow Diagram Level 3 Student Alert and Driver Response Flow (DFD3)	38
3.16 Entity Relationship Diagram (ERD)	39
3.17 Flowchart IOT Data Collection and Update Process	40
3.18 Flowchart Student Alert and Driver Response	41
3.19 Proof of Concept	42
3.19.1 Login Page for Users	43
3.19.2 Student Dashboard	44
3.19.3 Driver Dashboard	46
3.19.4 Admin Dashboard	47
3.20 Summary of Methodology	48
REFERENCES	50

LIST OF ABBREVIATIONS

Abbreviation Full Term

Agile	Agile Software Development Methodology
API	Application Programming Interface
APK	Android Package Kit
CD	Context Diagram
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
ESP32	Espressif Systems 32-bit Microcontroller
FIK	Fakulti Informatik dan Komputeran
FSM	Finite State Machine
FYP	Final Year Project
GPS	Global Positioning System
HTML	HyperText Markup Language
IDE	Integrated Development Environment
IoT	Internet of Things
IR Sensor	Infrared Sensor
IT	Information Technology
NodeMCU	Node Microcontroller Unit
NoSQL	Non-Relational Structured Query Language
PBSA	Priority-Based Scheduling Algorithm
PoC	Proof of Concept

Abbreviation Full Term

SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface
UAT	User Acceptance Test
URL	Uniform Resource Locator
UX	User Experience
UniSZA	Universiti Sultan Zainal Abidin

LIST OF TABLE

Table No.	Title	Page
Table 2.2.1:	Table of Existed Systems	15
Table 3.1:	Hardware Requirements	28
Table 3.2:	Software Requirement	29

LIST OF FIGURE

Figure No.	Title	Page
Figure 1.1:	Gantt Chart	10
Figure 3.1:	Agile Methodology	24
Figure 3.2 :	Framework	31
Figure 3.3 :	Context Diagram	32
Figure 3.4:	Data Flow Diagram Level 1 (DFD1)	34
Figure 3.5:	Data Flow Diagram Level 2 Collect Bus Tracking Data(DFD2)	35
Figure 3.6:	Data Flow Diagram Level 2 Locationa and Passenger Count (DFD2)	36
Figure 3.7:	Data Flow Diagram Level 3 Bus Tracking Update Flow (DFD3)	37
Figure 3.8:	Data Flow Diagram 3 Student Alert and Driver Response (DFD3)	38
Figure 3.9 :	Entity Relationship Diagram (ERD)	39
Figure 3.10:	Flowchart IOT Data Collection and Update Process	40
Figure 3.11:	Flowchart Student Alert and Driver Response	41
Figure 3.12:	Login Page for Users	43
Figure 3.13:	Student Interface	44
Figure 3.14:	Passenger Count Function	44
Figure 3.15:	Driver Dashboard	46
Figure 3.16:	Admin Dashboard	47

ABSTRACT

The Go-Campus Shuttle Management System is an intelligent transportation system designed to improve the operational efficiency of campus shuttle through the integration of Internet of Things (IoT) based devices, a mobile-based application for commuters, and a rule-based event-driven algorithm. The objective of this project is to improve the commuting experience of university students and staff by offering an app that will track shuttles in real time and passenger loads on shuttles. The system uses an ESP32 microcontroller interfaced with a GPS module and infrared passenger-counting sensors to obtain shuttle location and occupancy information in real time. This information is sent to the database (cloud) where it is stored and can be retrieved immediately. We use a rule-based event-driven algorithm, which assesses real-time sensor data based on pre-defined thresholds, thus allowing the system to independently trigger alerts automatically when specific conditions are met such as overcrowding or delays. Description: Development of mobile application with 3 main roles of user, students, drivers and admin. Shuttle location and seat availability to students, shuttle status notifications and status updates to drivers, and user account management and system operation logs to administrators. In the end, Go-Campus will enhance reliability of shuttle services, minimize waits and contribute to better, smarter transportation planning on campus.

ABSTRAK

Sistem Pengurusan Bas Ulang-Alik Go-Campus ialah penyelesaian pengangkutan pintar yang menyediakan pengintegrasian perkhidmatan bas kampus dengan teknologi IoT, aplikasi mudah alih, serta algoritma berdasarkan peraturan dan peristiwa. Penyelidikan ini dibangunkan untuk menyediakan alatan penjejakan bas secara masa nyata serta pemantauan kapasiti penumpang guna meningkatkan pengalaman perjalanan pelajar dan kakitangan. Sistem ini menggunakan ESP32 sebagai mikropengawal dan disambungkan dengan modul GPS serta sensor inframerah untuk mengira jumlah penumpang serta mendapatkan lokasi bas secara tali sawat. Data yang diperolehi akan dihantar ke dalam pangkalan data yang berada di awan untuk penyimpanan atau capaian aplikasi secara masa nyata. Bagi algoritma peristiwa, data yang diperolehi oleh sensor inframerah dinilai berdasarkan tahap secara berasaskan ambang yang ditentukan dan membolehkan sistem menyuarakan amaran secara automatik ketika berlaku kesesakan penumpang atau kelewatan bas. Aplikasi ini dibangunkan dalam tiga peranan utama, iaitu bagi pelajar, pemandu, dan pentadbir. Bagi pelajar, aplikasi ini menyediakan fungsi untuk melihat lokasi bas dan tempat duduk yang kosong. Manakala bagi pemandu bas, aplikasi ini akan memberikan notifikasi ke telefon pintar serta membuat kemas kini oleh pemandu berkaitan status bas. Bagi aplikasi yang dimiliki oleh pentadbir, terminal adalah untuk mengurus akaun pengguna serta pemantauan keadaan sistem. Kesimpulannya, Go-Campus telah berjaya meningkatkan keupayaan untuk menggunakan servis bas serta dapat mengurangkan masa menunggu untuk menunggu bas tiba.

CHAPTER 1

INTRODUCTION

1.1 Introduction

Apart from this, efficient transportation surely is one of the key enablers of a university for day-to-day operation. Despite this, students have had to deal with the problems of waiting long times, not knowing when shuttles will land as well as overcrowded shuttles and buses and waiting for quite a long time after getting on. To tackle these problems, a newly-implemented IoT-based system, Go-Campus, is proposed to provide real-time campus shuttle movement monitoring and passenger load monitoring.

It uses a GPS module for real-time tracking of the shuttles and sensors that track passenger entry and exit so the number of people in the shuttles can be known at any time. Using the system, students can check shuttle availability, location, and occupancy status prior to reaching the bus stop. Reserving the shuttle assists in avoiding needless wait periods and ensures that students do not enter a filled shuttle.

Combined with hardware (ESP32 / NodeMCU + GPS module + Sensors) and cloud + Dashboard, Go-Campus provides complete smart transportation solutions. In general, this system helps in better planning of travel, improved campus mobility, and an improved and reliable campus ride experience for the students.

1.2 Project Background

The problem associated with an efficient campus transportation system is a quite generic one, and it is even more aggravated in campus environments of large

universities where students rely more on shuttles to travel between hostels, faculties, and facilities. Traditionally, campus shuttles run without real-time traceability or passenger monitoring, leading to lengthy, unpredictable waits and overcrowded shuttles. Such problems can impact students routine and riding experience for both good and bad during peak hours includes more demands.

IoT technology can also be used to further improve campus transportation systems through automation and real-time data tracking. By using IoT-based solutions, it becomes easy to track real-time location of the vehicle as well as the passenger load while users can get the accurate and up-to-the-minute information. Answer to these problems has been determined by Go-Campus system which developed by integrating GPS technology and passenger detection sensors to monitor shuttle movement and campus occupancy in real-time. It sends this information to a cloud-based platform which could enable students to know whether shuttles are available and their current location before reaching the bus stop.

The Go-Campus system is an ideal fit for campus settings, where speedy transportation and time usage matter most. It reduces unnecessary waits, avoiding students getting on shuttles when they are full and helps to plan travel and maximises shuttle usage. Go-Campus which brings together individual hardware blocks like ESP32 or NodeMCU boards, GPS modules, and sensors with the cloud-based component-based block elements and a dashboard interface to provide holistic smart transportation solutions. In a nutshell, the system combines the campus mobility, enables more intelligent management of transportation, and students have a better commuting experience everyday

1.3 Problem Statement

Although shuttle bus services have been available on many university campuses, transportation management is still a primary concern area owing to the absence of real-time optimization and operational control. They are often left wondering about the shuttle arrival time, depends heavily on finding a bus and wondering if there is a seat for them to sit on resulting in people waiting in line for long periods of time, crowded buses, and overall bad service. Likewise, campus administrators have scarce accurate data to make sure shuttles are in service efficiently. These problems show the requirement for an intelligent and technology-powered shuttle management system that provides live tracking, passenger load monitoring, and real-time updates to enhance the overall commuting experience. Thus, the challenges identified above lead to the conception of a Go-Campus system which introduces the incorporation of IoT for tracking and provides a real-time integration with dashboard.

1.3.1 Lack Of Real-Time Shuttle Location Information

Currently, the campus shuttle system does not give students real time shuttle location or estimated arrival time. The shuttle schedule is typically static or only an estimate, and therefore does not reflect on-the-ground realities which could include traffic and delays. This leads to students waiting at shuttle stops with no indication of when exactly the shuttle will arrive. Causing a delay, students frustration and inefficiency, especially during prime academic hours when students are rushing to get to classes or events on campus.

1.3.2 Absence Of Shuttle Occupancy Monitoring

Another prominent challenge in the current campus shuttle system is the unavailability of real-time monitoring of shuttle occupancy. It never even occurred to me that students might have no way of knowing if a shuttle is full until it arrives at the bus stop. This often leads to overcrowding, discomfort, and even safety risks, especially during busy times of day. If a shuttle is full, a student must wait for the next to come, creating delays and impacting students' punctuality to classes, even exams or appointments.

1.3.3 Uncertainty In Shuttle Arrival And Location

But the campus shuttle does not provide students real time, accurate information from where to when it will arrive. Shuttles are typically scheduled at regular time intervals regardless of demand and traffic conditions. Consequently, students are often left guessing, waiting at shuttle stops, while still wasting their time and suffering inconvenience, especially in peak hours when on-time arrival at classes are mandated.

1.3.4 Inefficient Shuttle Management And Resource Allocation

Without this automated data collection on shuttle usage, including passenger numbers and shuttle movement patterns, the campus shuttle system also falls behind. Campus administrators are left unable to leverage current data to refine shuttle routing, scheduling, and frequency. This causes misallocation of resources, making shuttles crowded and others less full than they can be. As a result, shuttle services are made erratic and less productive to accommodate student movement.

1.4 Objectives

The prime focus of the project is to design an IoT- based campus shuttle monitoring system Go-Campus, where we can get info about the exact location of shuttle and the passenger occupancy status in the shuttle instantly. To this in turn means a solution that can effectively ensure less waiting time for students, stop crowding, and make overall travel planning way more effective. With GPS tracking, sensors that count passengers, and a cloud-based dashboard, the system both improves the commuting experience for students and offers campus administrators insights on how to improve shuttle routes and schedules.

To better understand existing behavior regarding campus shuttle operation and performance and where the trouble spots are. Shuttle schedules, passenger flow, and waiting time have been examined, along with some other constraints like a lack of real-time information or occupancy updates. Awareness of these issues makes sure that the Go-Campus system is able to solve genuine issues of students and campus management.

The aim is to create an IoT based shuttle monitoring system. This aim will be achieved through the development of a system comprising GPS MODULES, PASSENGER COUNTING sensors and a microcontroller (ESP32 or NodeMCU) which wirelessly capture the real-time data about shuttle location and occupancy status. It will send this data to a cloud platform for processing and storage.

To deploy mobile or web-based application dashboard for real-time tracking. Through the dashboard, students can learn the location, actual status and load status of the shuttle before they arrive at the bus stop. The dashboard can also be helpful

for campus management to monitor shuttle performance, analyze usage trends, and make data-driven choices on routes or scheduling.

For performance, accuracy, and effectiveness evaluation of the system. This includes testing real-time data transmission, sensor accuracy, and dashboard reactivity. The assessment will measure how it implements the wait time minimization, crowding mitigation, as well as shuttle service efficiency throughout the campus.

1.5 Scope

This is further expanded as venture in the project scopes, as it was only developing the Go-Campus smart shuttle monitoring system to assist the shuttle bus transportations in the university campus. This incorporates Internet of Things (IoT) technologies for real-time tracking of both shuttle locations and passenger loads. The stack involves GPS modules for determining the position of a vehicle, infrared sensors for counting passengers, and cloud computing platforms for persistent storage and synchronization of records. It will integrate shuttle status, and occupancy for students, while providing administrators with a dashboard with which they can manage system settings and take note of shuttle performance. This project will not focus on vehicle control, route optimization (beyond predefined campus routes), or other areas not related to monitoring campus shuttle services and delivering information to riders. It is supposed to be piloted at a controlled campus to improve commuting efficiency, minimize ambiguity around passenger pickup, and simplify transportation management.

1.5.1 User Scope

The Go-Campus system is designed for use within a campus environment to provide real-time shuttle location and occupancy information. The system will cater to the following user types.

Admin (Overall System Manager): This user is responsible for maintaining the system and managing all the shuttle details, tracking the locations and number of passengers in shuttle, changing routes or schedules, and creating the reports for the usage and performance of shuttle

Shuttle Driver / Operator : Drives shuttles with IoT devices, they ensure that devices are working perfectly, can see active routes or inform the admin for any system/device failures.

Student / Public User (Passenger): Primary users who use the shuttle to get real time shuttle information and plan their travel. Students see the famous blue dot of exactly where shuttle are, what their availability is, and occupancy status, but they have no edit or other administrative features.

1.5.2 Application Scope

The Go-Campus system focuses on improving campus shuttle efficiency by providing real-time monitoring and data-driven insights.

System Architecture: Development of an IoT based system which integrates GPS and passenger counting sensors with a microcontroller (ESP32/NodeMCU) to track the shuttle in real-time, and implement a passenger counting mechanism.

Dashboard Development: Develop a web or mobile dashboard for students to see locations and occupancy status of shuttles to view travel options and for admins to monitor shuttles, update their route, and view usage data.

Campus Deployment: It designed to deploy the system around the campus and cover the area with permanent Wi-Fi/cloud connectivity.

Limitations: This phase doesn't touch payment systems, ticket booking, or integration with external public transport networks. It can only track shuttles and passengers on campus.

1.6 Limitation of Work

The Go-Campus system, while providing an innovative solution for campus shuttle monitoring, has the following limitations.

Web Dependency: The availability of active and steady Wi-fi or cellular connection is required to allow real-time shuttle monitoring and dashboard changes. In a poorly connected region, it may take time for the system to reflect the current location of shuttle and its occupancy, leading to low system reliability.

Power Supply Needs: The hardware components such as the GPS module, sensors, and ESP32/NodeMCU will be powered internally inside the shuttle. That limits deployment in shuttle with erratic power or during power loss.

Passenger Counting Accuracy and Maintenance: If two students board or exit at the same time, it can create confusion for passenger counting sensors, so they require careful setup and occasional recalibration to accurately evaluate passenger volume.

Less Adaptability to Environment: Real-time location updates and mapping can be less accurate in places that are tall as buildings and tunnels, which can hinder the GPS or find out the places from one point to another in adverse weather conditions.

Scalability Limitations: While the system has been successfully used, especially for campus-scale deployment then but may need further modifications to adapt to larger campuses, fleets of shuttles and external transport systems.

1.7 Expected Result

The Go-Campus system is expected to deliver the following key results.

More Efficient Shuttle Service: Real-time shuttle location and passenger occupancy info will allow students to plan their trips accordingly, instead of waiting at shuttle stops for no reason at all.

Improved Operational Control over Campus Management: A supply of real-time utilization data, along with an in-cloud dashboard from which admins will have the ability to monitor shuttle routes and passenger load as well. This will help to manage shuttle operations, adjust routes, and optimize schedules accordingly.

Less Crowding & Better Resource Utilization: The system will eliminate stalking, add to the user comfort, and improve the utilization of shuttle resources by letting the system know of the real time occupancy in the shuttle especially at peak hours.

1.8 Gantt Chart

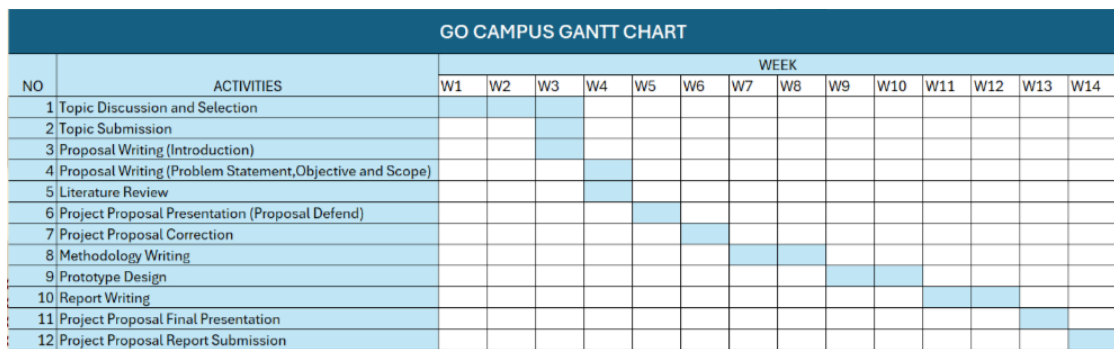


Figure 1.1: Gantt Chart

1.9 Summary of the Chapter

Go-Campus is a project which works on the development of a IOT-based campus shuttle monitoring system which provides realtime information on shuttle location and status of passenger occupancy. The system uses GPS modules, passenger counting sensors, and a microcontroller (ESP32/NodeMCU) to gather information on the shuttle movements and occupancy and sends this information to a cloud platform for visualization in a web/mobile dashboard. It enables students to be more efficient with their travel, cutting down on wait times and preventing overcrowded shuttles.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

The rise of smart campus has triggered the need for efficient and intelligent transportation systems in the university surroundings. Although campus shuttle services support daily mobility for students and staff, many campuses are still using traditional shuttle systems without real-time tracking and passenger information. The result? Long lines, more crowds, and inefficient shuttling. However, research has shown that IoT can enhance operational efficiency and user experience by enabling real-time monitoring, data-driven decision-making, and enhanced service transparency (Sujitha et al., 2020). The literature review discusses IoT-based transportation systems, GPS vehicle tracking, passenger load monitoring, and cloud-based dashboard to lay a basis for the Go-Campus system.

2.2 Smart Transportation and IoT-Enabled Campus Mobility

Smart campus transport solutions are set up to improve mobility powered by mobility technology coupled with technologies such as wireless communication, sensors and analytics in real time embedded into shuttle operations. These systems enhance passenger experience by providing the ability to track shuttles in real-time and providing an estimated time of arrival, allowing passengers to better plan their trips. Legacy campus shuttle services typically struggle with inflexible shift planning and a lack of transparency, ultimately resulting in variable levels of service and underutilization of resources. When real-time data is available to the users of roads and

administrators, intelligent transport solutions (ITS) reduce congestion and enhance punctuality while further facilitating sustainability-oriented mobility initiatives (Karthik et al., 2023; Zhang et al., 2024). This is why the implementation of smart monitoring platforms like Go-Campus is becoming salient over different campuses of the world.

Major Role of Internet of Things (IoT) towards Smart Transportation Internet of Things (IoT) plays a vital role behind the enabling smart transportation and have become the network of physical devices that automatically collect, send, and process data in real-time. A typical IoT-based transport system includes an integrated architecture of sensors, microcontrollers, communication modules, and cloud platforms for long term monitoring and control. According to Sujitha et al. (2020), studies have shown that integrating IoT minimizes uncertainty among passengers by providing real-time information regarding the movement of the vehicle. Moreover, the IoT based sensing systems are more robust and scalable as the data is collected automatically, without needing human intervention (Mohamed et al., 2023). IoT is the underlying technology behind campus transportation transmitting shuttle location and passenger load information in real time to dashboards, the basis for Go-Campus and similar systems.

2.3 Real-Time Vehicle Tracking and Passenger Load Monitoring

Real-time tracking is one of the major contributions of smart transportation systems and as a result, using GPS technology has been popular to identify location, speed, and route information of a car. Success stories include adopting GPS based vehicle tracking in public buses, delivery fleets, and campus shuttles for improved operational tracking and visualization of routes. Much of research indicates a significant decrease in waiting time by monitoring a shuttle via GPS, for passengers can track the shuttle through a

mobile or web application (Karthik et al., 2023). GPS accuracy degrades in areas with many tall buildings or weak signals, but it is still the best and cheapest real-time solution available, and the heart of every Go-Campus vehicle.

Apart from tracking, keeping an eye on passenger load is also important in public transport so that it does not get overcrowded, ensuring passenger comfort. Different methods have been studied like load sensing, infrared sensing, computer vision and machine learning methods. They have been extensively used as they offers more or less optimal trade off among cost, accuracy, privacy and real-time performance (Mohamed et al., 2023) especially sensor-based approaches, for which infrared (IR) sensors are often used. The camera-based solutions can deliver comprehensive occupancy statistics, but they have privacy issues and they require higher computational power. Therefore, the approach used in Go-Campus is applicable and feasible also on campus shuttle environments, thus IR sensor based passenger counting would seem more suitable passenger counting method.

2.4 Cloud Computing , Dashboards and User Applications

Moreover, cloud computing is essential in IoT-based transportation systems as it facilitates the storage, processing, and accessibility of shuttle data in real-time. Platforms like Firebase and ThingSpeak are widely employed for the effective processing of GPS and sensor data. To facilitate data-driven decision-making, administrators can, via cloud-based dashboards, monitor vehicle performance, passenger flow, and operational status from outside of the vehicle (Lin and Hsieh, 2022). They also provide transparency to passengers who can see map-based real-time shuttle location, real-time occupancy status via intuitive interfaces. Cloud integration

in Go-Campus enables automatic syncing between the shuttle hardware as well as the mobile or web dashboard.

Mobile and web applications are the primary interface between the users and the smart transportation services. They offer a variety of live information, ranging from visual maps to occupancy indicators, enabling passengers to plan their travels accordingly. The dashboards are intuitive and user-friendly as suggested by research that easily accommodates the adoption and effectiveness of smart transport systems (Zhou et al., 2024). Mobile-accessible platforms are particularly paramount in campus settings, where students are likely to use smartphones in their typical daily routines. Building on this idea, Go-Campus provides a read-only access to real-time shuttle information for students while administrators can maintain configuration/usage analysis trends.

2.5 Rule-Based Event-Driven Monitoring with Threshold Logic

Go-Campus uses a rule-based event-driven mechanism that blends threshold logic for effective real-time decision making. This method constantly fetches real-time data from IoT sensors, like the GPS location or number of passengers in the bus, and check it against predefined conditions such as maximum bus capacity or tolerable arrival delays. Based on specific rules with regards to passenger load exceeding threshold or the delay going beyond the prescribed waiting window, the system automatically triggers appropriate events. Such events may be updating shuttle status, real-time notifications to students, sending alerts to drivers in vicinity. This enables Go-Campus to automatically respond to changing transportation conditions in real time without having to constantly monitor it; facilitating a smarter and more dynamic campus shuttle environment.

2.6 Challenges and Limitations of Existed Systems

Table 2.2.1: Table of Existed Systems

No	Author / Year	Title	Technique Used	Advantages	Disadvantages
1	P. Subramaniam, et al. (2019)	Passenger and Luggage Weight Monitoring System for Public Transport	Rule-Based Sensor Calculation (Load-cell weight sensing linked to microcontroller logic)	This system is capable of monitoring vehicle load capacity in real time, which helps prevent overloading and enhances passenger safety. The automated data collection process minimizes the need for human intervention, making the system efficient and suitable for continuous operation in public transportation environments.	Although the system can measure total vehicle load, it does not provide an exact passenger count, which limits its usefulness for occupancy-based decision making. Additionally, it does not support vehicle location tracking, and its accuracy may vary depending on sensor calibration and environmental conditions.
2	Sujitha K., et al. (2020)	IoT Based Bus Arrival Time Prediction Using Artificial Neural Network for Smart Public Transport System (SPTS)	Artificial Neural Network (ANN)	This system uses machine learning techniques to predict bus arrival times, helping to reduce passenger uncertainty and waiting time. Real-time tracking features further improve the user experience by allowing passengers to plan their journeys more effectively.	The system does not include passenger load or seat availability monitoring, which limits its ability to address overcrowding issues. It also requires stable internet connectivity and accurate datasets for model training, making implementation more complex compared to rule-based tracking systems.
3	Karthik S., et al. (2023)	Bus Tracking and Arrival Prediction System	Rule-Based GPS Tracking (Location update logic via GPS → Server → App)	The system enables passengers to track the live location of buses through a user-friendly mobile or web dashboard. By providing real-	This approach does not support passenger occupancy detection, meaning users cannot determine seat availability before boarding.

				time location updates, it improves operational efficiency and reduces unnecessary waiting time at bus stops.	Furthermore, tracking accuracy depends heavily on GPS signal strength, and the system provides limited analytical insights for campus transportation management.
4	Mohamed A., et al. (2023)	Passenger Occupancy Estimation in Vehicles: A Review of Methods	Computer Vision / Machine Learning / IR Sensors	This system provides valuable insights into multiple passenger counting techniques, helping researchers and system designers identify the most efficient detection methods. It supports optimization of public transportation systems by comparing the accuracy and feasibility of different approaches.	Some passenger counting methods, particularly camera-based systems, are costly and raise privacy concerns. Sensor-based techniques may also experience miscounts during peak movement periods, which can affect overall system accuracy.
5	Zhang, H., Li, W., & Wang, Y. (2023)	Research on Dynamic Scheduling and Route Optimization Strategy for Flex-Route Transit	Dynamic scheduling model + optimization algorithms for real-time routing	The system supports real-time flexible routing by adjusting shuttle routes based on passenger demand and traffic conditions. This adaptability makes it suitable for campus shuttle scenarios where demand fluctuates throughout the day.	Implementation details are often limited, making real-world deployment challenging. The system also requires a reliable IoT and real-time data infrastructure, which may not always be available in smaller environments.
6	Lin, J., Chen, L., & Wu, X. (2022)	Multicriteria Route Planning for In-Operation Mass Transit Systems	Heuristic spanning tree algorithms for multi-criteria route planning (flow, fuel, transfers, welfare)	This system optimizes multiple objectives simultaneously, such as operational efficiency, passenger convenience, and	The system is typically designed for large-scale public transportation networks and may require adaptation for smaller shuttle fleets.

				cost reduction. It is useful for achieving balanced transportation planning decisions in complex transit networks. This system optimizes multiple objectives simultaneously, such as operational efficiency, passenger convenience, and cost reduction. It is useful for achieving balanced transportation planning decisions in complex transit networks.	Additionally, it is computationally intensive compared to simpler heuristic-based approaches.
7	Zhao, P., Liu, Q., & Wang, R. (2022)	Transit Dynamic Operation Optimization Using Stop-Skipping Control and Local Route Optimization	Mixed strategy: stop-skipping + local route optimization to reduce passenger wait & operational costs	This approach reduces travel and operational costs by dynamically adapting to changing passenger demand. It can be tailored for single-line campus shuttle routes, making it relevant for university transportation systems.	The system is often limited to single-line routes and requires accurate real-time passenger data to function effectively. Without reliable data input, scheduling efficiency may be reduced.
8	Kim, S., Park, H., & Lee, D. (2023)	Service Management of Employee Shuttle Service Under Dynamic Demand	Dynamic fleet management model (optimization under variable demand)	This system efficiently handles variable passenger demand by optimizing shuttle allocation and scheduling. It is directly applicable to campus shuttle systems where travel patterns change throughout the day.	Most implementations focus on employee shuttle services, meaning adjustments may be required to suit student-based transportation patterns. The system also relies heavily on accurate demand prediction to maintain efficiency.

9	Zhang, Y., Chen, F., & Liu, H. (2022)	Integrated Multi-Objective Optimization for Dynamic Airport Shuttle Bus Services	Multi-objective mixed-integer linear programming (MILP) for route, frequency, and station choice	This system addresses multiple constraints through multi-objective optimization and adapts dynamically to time-varying passenger demand. The approach can inspire advanced optimization strategies for campus shuttle systems.	The system is computationally intensive and designed for large-scale environments such as airports. As a result, it may require simplification to be practical for campus-scale shuttle routes.
10	Sun, Q., Hu, D. W., Chien, S., et al. (2022)	Dynamic Bus Routing Optimization for Demand-Responsive Feeder Transit Considering Stochastic Bus Arrival Time	Mixed-integer optimization + heuristic (Genetic Algorithm + neighbourhood search)	Handles real-time demand dynamically. Models stochastic bus arrival times, improving realism. Reduces passenger waiting time and operational costs.	Requires detailed arrival time and demand data. Heuristic may not guarantee global optimality. Designed for feeder transit; may need adaptation for campus shuttle.

A passenger and luggage weight monitoring system was proposed to address the problem of overloading in public buses, which can compromise road safety. The system uses sensing technologies such as load cells and wireless sensor networks to measure the total weight of passengers and luggage in real time, with data transmitted via IoT and cloud computing for monitoring and analysis. Although this approach provides useful information on vehicle load conditions, it focuses on aggregate weight rather than exact passenger count and may not be suitable for applications where occupancy details are critical, such as campus shuttles that require accurate seat availability information (Bwalya et al., 2019).

An IoT-based bus arrival time prediction system employing artificial neural networks (ANNs) has been developed to improve the accuracy of travel time estimation in smart public transport systems. By learning from historical data and real-time position data

through GPS, the system reduces waiting time uncertainty for passengers. Studies have shown that machine learning models such as ANN outperform traditional methods in predicting arrival times with lower error rates, enhancing passenger experience and operational reliability. However, this type of model typically requires substantial historical datasets and strong internet connectivity, limiting its practical deployment in smaller or under-connected environments (Noor et al., 2020).

Bus tracking and arrival prediction systems use GPS modules and mobile or web applications to provide live information on bus locations and estimated arrival times. These systems enhance user experience by offering up-to-date travel information and reducing uncertainty at stops. The integration of real-time tracking with intuitive dashboards helps improve route visibility and decision making for commuters, but such systems often do not include mechanisms for monitoring passenger load or optimizing schedules based on occupancy, which are important for preventing overcrowding in transport services.

Occupancy estimation in public transport vehicles has been the subject of extensive research, with systems ranging from invasive to non-invasive methods. A comprehensive review highlights that invasive techniques typically yield higher accuracy, while non-invasive methods such as infrared and camera-based systems maintain passenger privacy but face challenges like environmental interference. Understanding the strengths and limitations of different approaches is crucial for designing systems that balance accuracy, cost, and user privacy, particularly in contexts where detailed passenger counts are necessary for service planning and safety (Kuchár et al., 2023).

Flex-route transit systems dynamically adjust vehicle routes and schedules based on real-time passenger requests and operational conditions, allowing more responsive service than fixed-route systems. Research in this area has focused on interactive decision models that account for passenger travel preferences and traffic conditions. The results demonstrate that such dynamic strategies can improve service performance and adaptability by optimizing dispatching decisions, although the complexity of planning under variable demand remains a research challenge (Zhang et al., 2024).

Mass transit route planning methods that incorporate multiple criteria, such as passenger waiting times, vehicle mileage, and load distribution, have been proposed to enhance transit operations. Multi-objective optimization models balance operational efficiency with service quality, often employing heuristic or metaheuristic techniques to find feasible solutions under real-world constraints. These approaches help transit agencies achieve balanced performance across diverse operational targets, indicating potential insights for campus shuttle scheduling, though adapting such models for smaller scale or single-route systems requires simplification (Han et al., 2025).

Dynamic operation optimization models that combine strategies such as stop-skipping and local route adjustments have shown promise in improving the operational efficiency of single-line bus services. By reducing unnecessary stops and optimizing the route locally, these models can shorten travel time, improve vehicle speeds, and enhance passenger satisfaction. Hybrid algorithms that integrate genetic algorithms with heuristic methods have been used to solve these optimization problems more effectively, demonstrating improved operational performance compared to traditional scheduling methods (Zhou et al., 2024).

Employee shuttle services operating under dynamic demand conditions have been studied to understand how real-time scheduling and flexible routing can adapt to changing travel patterns. These systems focus on maintaining service quality while adjusting resources to meet demand variations, often incorporating feedback mechanisms and predictive models to enhance efficiency. While primarily targeted at workplace environments, the insights from such service management strategies can inform similar solutions for student shuttle systems that experience varying peak and off-peak travel demands.

Airport shuttle services face the challenge of simultaneously optimizing route design and departure frequency in time-varying demand environments. Multi-objective mixed-integer programming models have been developed to minimize passenger ride and waiting times while also reducing operational costs, yielding Pareto-optimal solutions that balance competing objectives. Although focused on large-scale airport transport, these optimization frameworks highlight the importance of multi-criteria decision making in shuttle service design, which could inspire similar methods for campus shuttle systems under different constraints (Wei et al., 2022).

Demand-responsive feeder transit systems that incorporate stochastic arrival time considerations use dynamic routing optimization to improve service reliability and reduce uncertainty. These systems model the probabilistic nature of arrival and travel times to enhance route flexibility and responsiveness to real-time conditions. By integrating predictive analytics with operational decision making, such approaches can significantly reduce passenger waiting times and improve overall service efficiency, though they require robust data integration and advanced computational methods for real-time execution.

2.7 Summary of Literature Review

In summary, existing research demonstrates that IoT-based transportation systems incorporating GPS tracking, passenger load monitoring, and cloud dashboards can significantly improve shuttle service efficiency and user satisfaction. However, many current systems either focus solely on location tracking or rely on complex and expensive technologies. The Go-Campus project addresses these gaps by combining affordable IoT hardware, real-time data transmission, and a user-friendly dashboard tailored specifically for campus environments. This literature review provides a strong foundation for the methodology and system design presented in the following chapter.

CHAPTER 3

METHODOLOGY

3.1 Introduction

The Go-Campus project workflow serves as a methodology to develop, test, and assess an IoT-based campus shuttle tracking and passenger load monitoring system. This system makes use of the IoT technology in which, we have GPS modules which will track the location of the shuttle, the ESP32/NodeMCU microcontroller, passenger counting sensors, cloud platforms and a web/mobile dashboard, It enables real-time shuttle tracking location and occupancy information. The methodology is highlighted around several development phases: requirement analysis, software development, testing and evaluation, to assure that the system meets the project objectives: to improve efficiency of shuttle service, to reduce waiting time for students, to avoid over-crowding of shuttle buses, as well as to provide data for better campus transportation management decision .

3.2 Agile Methodology

The Go-Campus project use the Agile methodology, which is an approach to software development under which requirements and solutions evolve through the collaborative effort of self-organizing and cross-functional teams. An example of where Agile is well aligned is with IoT-based projects like Go-Campus, where hardware and software components need to be integrated and honed, on both sides, in light of test results and user feedback.

In this project, we use Agile by being iterative: in short development cycles (called sprints). In each sprint, the system tools focusing on items such as shuttle location, passenger load, dashboard, etc. are developed, tested, and reviewed. It enables the identification of problems early on and the ability to enhance in stages.

Utilizing Agile principles like regular testing, iterative development, and constant feedback allows for the Go-Campus system to be shaped and designed in a manner that best suits the users' needs. It enables collaborative work and a successful system output that provides useful real-time data, advanced user experience, shuttle management on campus.

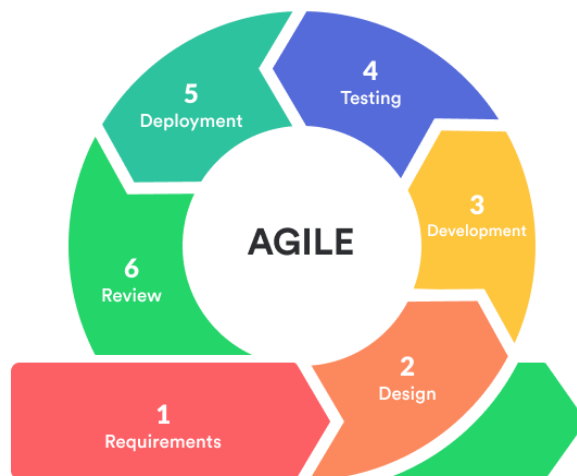


Figure 3.1: Agile Methodology

3.3 Requirement Analysis

Phase one investigates and collects the required system requirements for the Go-Campus project. On this stage we describe functional and technical specifications to support campus shuttle tracking in (near) real time and also passenger load monitoring.

One must also know what different groups of users have in terms of needs and the constraints to operate the system.

System Administrators, Shuttle Drivers and Students are the main users of the system. The role of the users and their interaction with the system are analyzed in order to ensure that the system design will be able to meet their particular needs. Students, for example, will need real-time data about shuttle locations and occupancy, while administrators will want to monitor and report on shuttle operations.

At this stage, basic specifications such as hardware selection (GPS module, passenger counting sensor, and ESP32 / NodeMCU microcontroller), software architecture (cloud platform and web or mobile dashboard integration), and real-time data transmission methods are defined. The following requirements are set in order to maintain reliable system operation, setup accurate data monitoring, and better support for campus shuttle management.

3.4 Software Development

After the system requirements, the software development process starts. This part of the project includes the actual design and development of the cloud platform and user interface for tracking shuttle information in real-time. A dashboard is created on the web/mobile with an interface for students/admin to see the location of shuttle buses & number of passengers. The cloud platform (Firebase, ThingSpeak, etc) where the data sent from the GPS module and passenger counting sensors installed on the shuttle are stored, processed and managed. Real-time updates followed by making it available to users through the dashboard. Lightweight messaging protocols manage the communication between the hardware and the cloud for the rapid and reliable transmission of the data point. It is a reliable, scalable and user-friendly software system that offers continuous monitoring of movement and passenger load of the shuttles.

Enabled with real-time updates and visual discovery of data, the system offers a robust shuttle management solution and contributes towards enhancing a seamless student commuting experience.

3.5 Testing

Testing is done to make sure that all the components of the Go-Campus system work correctly and in sync with each other. During the development process, different kinds of testing are performed to ensure the reliability and performance of the system.

Unit testing :This is performed to ensure that individual components such as GPS module, passenger counting sensors, ESP32/NodeMCU microcontroller, Dashboard interface etc. work as expected and in isolation.

Integrate testing : This is executed to ensure a good communication between system components. Being able to send data from the sensors and GPS module to the cloud platform, and be able to pull this info & display it live onto a website or mobile dashboard.

System testing: Tests the integrated Go-Campus system to see whether it meets the performance requirement. This includes measuring how accurately shuttles are tracked with respect to their location, passenger occupancy (and whether the readings are reliable), the dashboard responsiveness, and whether both are updated in real-time. Both in the lab and when the systems are integrated and running on campus shuttles, the system kept performing without any problem

3.6 Evaluation

The third and final phase, the evaluation phase assesses Go-Campus system — performance, reliability, and scalability. Evaluation: The system is evaluated in terms of providing accurate real-time shuttle location updates, reliable passenger load monitoring, and data transmission to the dashboard interface in a timely manner. It also evaluates the performance of the system under both normal and peak usage conditions. Evaluate system usability, clarity of displayed information, and overall satisfaction from students and administrators through user feedback. This feedback is used to assess if the system decreases waiting times, crowding in the stations and assists in trip planning. Moreover, the evaluation process pinpoints likely opportunities for improvement, be it sensor accuracy, dashboard features, or additional shuttle support. This results in exposing future system improvements and the possibility of scaling up to larger campus scope.

3.7 System Requirement

The following section specifies the system requirements that are essential for the development and deployment of Go-Campus smart shuttle monitoring system. System requirements specify the necessary hardware, software, and network infrastructure to support the operation of the proposed solution. Their knowledge helps direct the requirements needed to provide, access, and process data adequate for a real-time campus transportation system, including wireless communication, hardware, and integration with cloud and dashboard access. It is vital to identify these requirements to ensure that Go-Campus delivers user needs and operates consistently and reliably throughout its lifetime.

3.7.1 Hardware Requirements

Table 3.1 below shows some of the hardware used to run GoCampus system.

Table 3.1: Hardware Requirements

No.	Hardware	Description
1	ESP32 / NodeMCU	Acts as the main microcontroller that processes data from sensors and GPS modules. It also handles communication with the cloud platform via Wi-Fi.
2	GPS Module	Used to track the real-time location of the shuttle by collecting latitude and longitude data. This information is sent to the cloud for live map display.
3	Passenger Counting Sensors(IR Sensors)	Detect passengers entering and exiting the shuttle to calculate the current occupancy level and available seats.
4	Power Supply	Provides continuous power to the microcontroller, GPS module, and sensors during shuttle operation.
5	Connecting Wires and Breadboard / PCB	Used to connect and assemble the hardware components securely inside the shuttle.

3.7.2 Software Requirements

Table 3.2 below shows some of the software used to run BinWatch System.

Table 3.2: Software Requirement

No	Software	Description
1	Microsoft Word, Google Docs, Canva	Used for documentation, reports, and creating visual content like infographics.
2	Google Chrome, Microsoft Edge, Google Scholar	Web browsers for research and accessing academic papers via Google Scholar.
3	Arduino	IDE used for programming the NodeMCU microcontroller for sensor data processing.
4	Android Studio	Development environment for creating the mobile app that interacts with the BinWatch system.
5	Firebase	Cloud platform for real-time data storage and management, handling sensor data and notifications.
6	Draw IO	Tool for creating system architecture and flow diagrams for the project.

3.8 System Design

Let us walk through the key diagrams to illustrate Go-Campus system design. The Overview context diagram gives a high-level view of the system, demonstrating how it interacts with external entities for example, students, drivers of the shuttles, and the cloud platform Data Flow Diagram (DFD) Arrow shows the flow of data within the system from where the data is being captured by GPS modules and passenger counting sensors to when ESP32/NodeMCU processes the data and sends it to the cloud and finally and android app receives real-time updates on shuttle's location and how full it is. The entity-relationship diagram (ERD) represents the database design, displaying how some of the main entities (Shuttles, Users, and Trips) are related. Lastly, the Flowchart describes the process flow which displays the sequential flow of actions from detecting shuttle movement, counting passengers, then generating notifications, and updating the respective data. When taken together they show the architecture, data flow and operational processes of the system.

3.9 Framework

The Go-Campus Go system Framework .A comprehensive solution for providing shuttle tracking and passenger monitoring. The system comprises of GPS Modules and passenger counting sensors in shuttles which are received by an ESP32/NodeMCU microcontroller. It consists of a microcontroller that receives the sensor data and sends it to a cloud-based platform (for example Firebase or SQL) in which the data is stored and accessed in real time. The mobile app and dashboard also allows students and administrators to track shuttle locations and occupancy levels, sending student alerts for shuttle arrivals or available seats. It grants extreme admin control for the system,

while students are kept in the loop with instant notifications in a timely fashion to prepare for their trips. This framework enables tracking in real-time, allows operational efficiency, and contributes to a hassle-free campus shuttle experience.

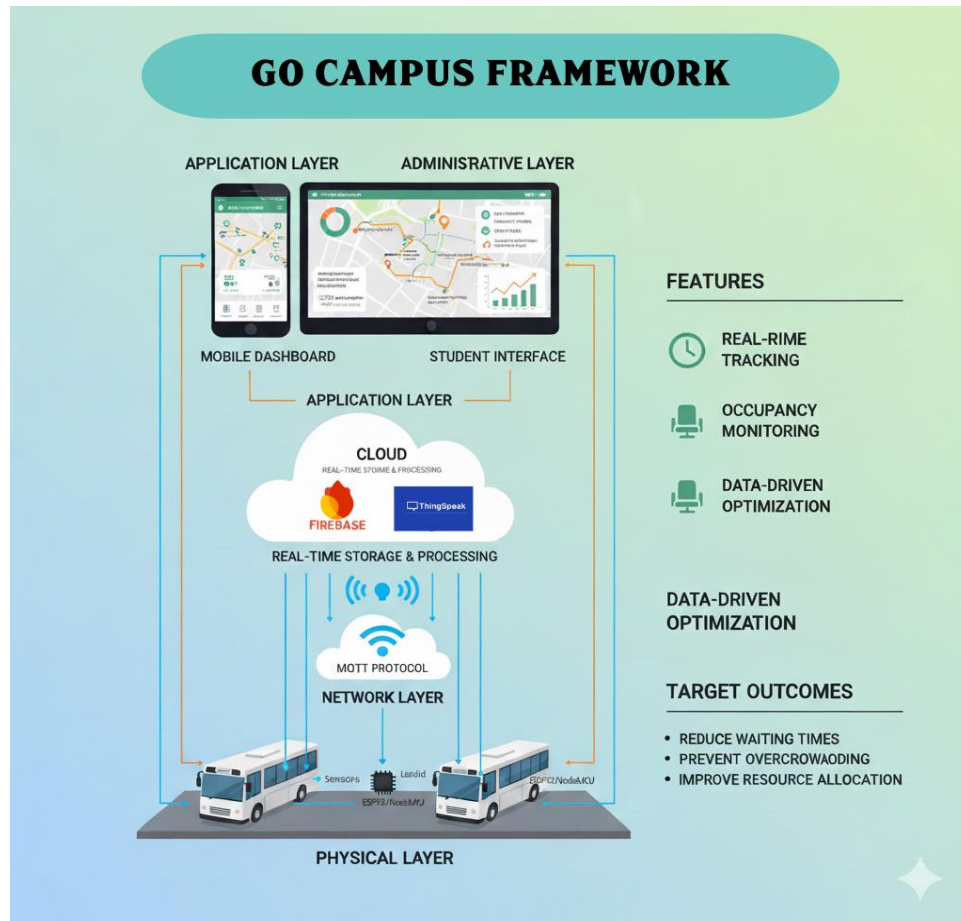


Figure 3.2 : Framework

3.10 Context Diagram

The Context Diagram provides a high-level perspective of the Go-Campus system by relative system boundary and explaining connections between Go-Campus and the external entities. Here the four main external actors are . Student, Driver, Admin, and an IoT Tracker. This system is powered by the IoT Tracker transmitting real-time

shuttle location and passenger count data. Students use the system to automatically pull real-time data on shuttles and submit alerts in case of a detected incident of delay or overcrowding. Alert notifications are received by the drivers who return Shuttle Status. The administrators access the system for managing the user accounts, monitoring system activities, and using operational data. This is a simplified view of the entire system as it only shows what is getting exchanged externally, and you need this to understand the scope of the system and what are all the systems its gonna interact with.

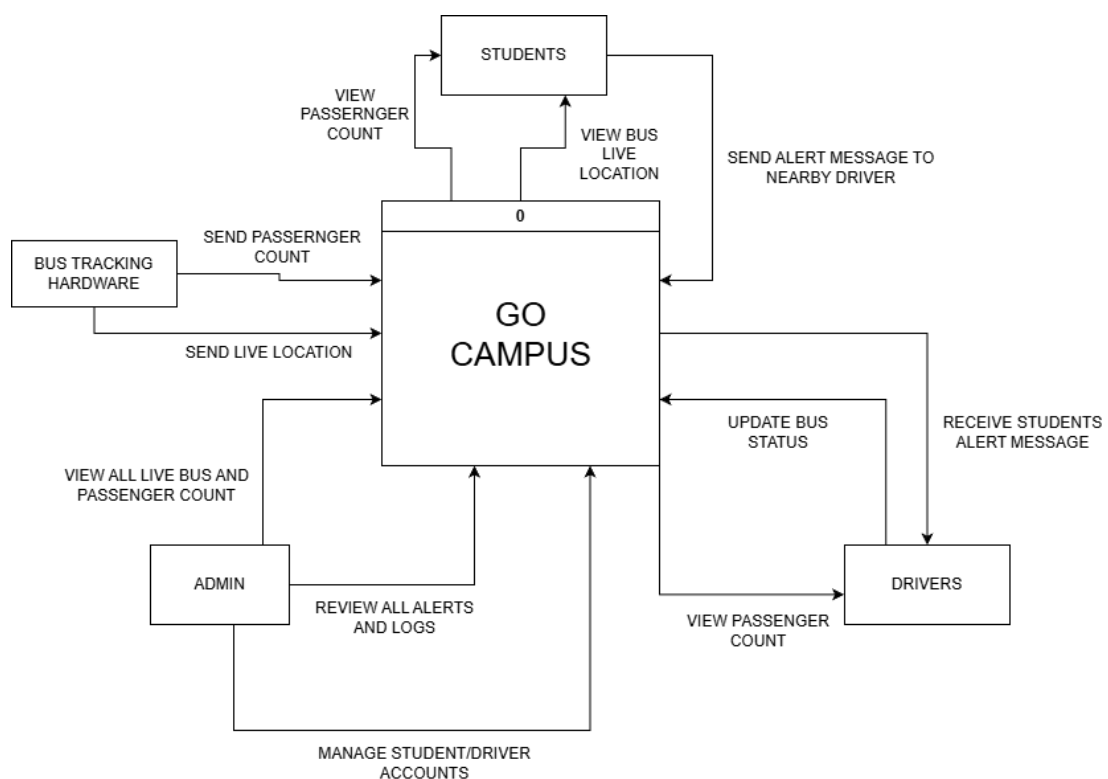


Figure 3.3 : Context Diagram

3.11 Data Flow Diagram Level 1 (DFD1)

Data Flow Diagram Level 1 visualizes the major processes, data flows, and data stores to show a functional decomposition of the Go-Campus system. These core processes

contain functions such as querying shuttle tracking data, displaying shuttle information to the students, processing the alert notifications, updating shuttle status, managing user accounts, and system log monitoring. Operational data can be persisted in data stores like Bus Information Store, Student Data Store, Driver Data Store and Alerts & Logs Store. The below diagram shows how the data that is coming from IoT devices is processed, stored and then made available for various users by making sure that each role gets the right information they require in the appropriate form. Level 1 DFD is a mid-level functional overview identifying all the major processes across the context diagram.

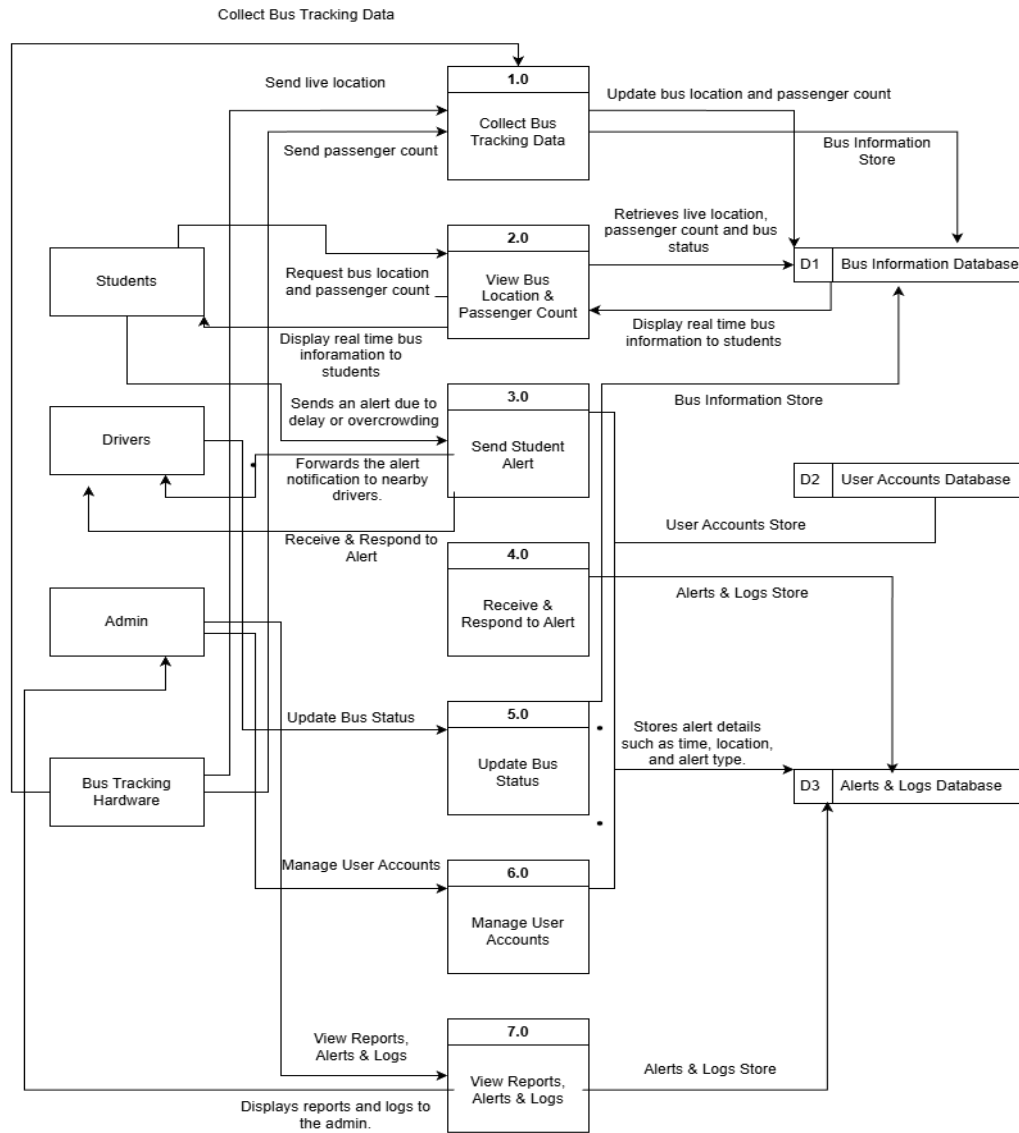


Figure 3.4: Data Flow Diagram Level 1 (DFD1)

3.12 Data Flow Diagram Level 2 Collect Bus Tracking Data (DFD2)

Context Diagram As DFD Level 2 for the bus tracking process portrays the detailed view of the way how unfiltered information of the sensors are processed in the system. Values of the GPS coordinates and passengers in a vehicle monitored by the IoT Tracker are sent toward the system for data validation in order to ensure the accuracy and integrity of the data. After validation, data is formatted and persisted in Bus Information

Store. These steps facilitate real-time updates on shuttle location and occupancy levels; both are critical for real-time tracking. This decomposition exposes something internal to the processing steps that are not at DFD Level 1, but are essential for reliability.

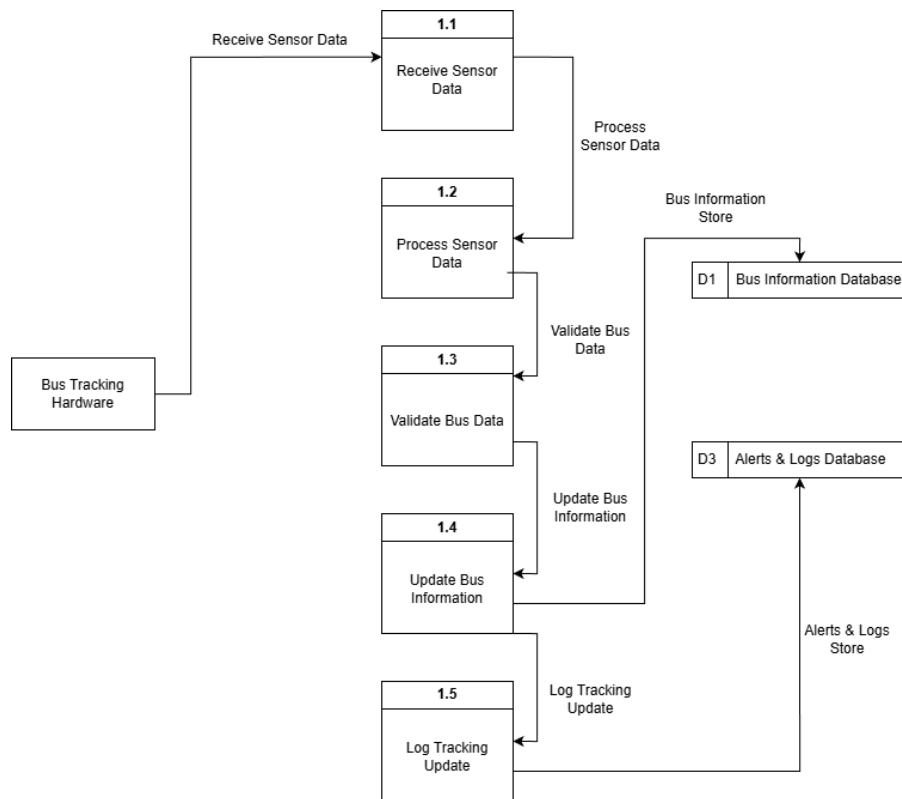


Figure 3.5: Data Flow Diagram Level 2 Collect Bus Tracking Data(DFD2)

3.13 Data Flow Diagram Level 2 View Bus Location and Passenger Count (DFD2)

This DFD Level 2 is specific to the process of retrieving data related to students. The system reads data from Bus Information Store. When a student requests shuttle information from the mobile application. The collected data is being processed and displayed in a more human-readable format that helps students to view current location

of shuttle, passenger load, and operational status (running or not). Here, system responsiveness and usability are taken into a high consideration so as to serve the students by providing correct information in little time.

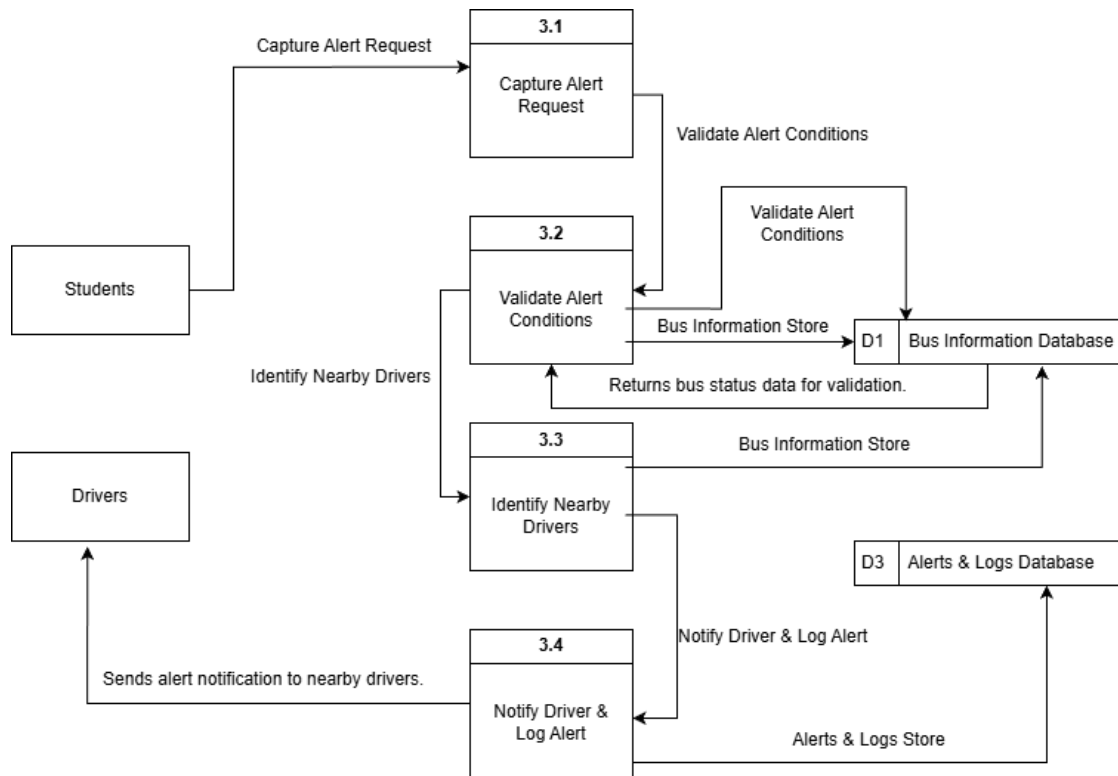


Figure 3.6: Data Flow Diagram Level 2 Location and Passenger Count (DFD2)

3.14 Data Flow Diagram Level 3 Bus Tracking Update Flow (DFD3)

DFD Level 3 provides even more detail about tracking the bus, showing the workflow that occurs inside. The incoming data from sensors, its validation, processing, and storage into the database. When the updated details are saved, they are instantly made accessible to all other processes of the system like student viewing and alert review. The detail level serves to demonstrate the nature of the requirement that data be updated in real time, but it also shows how the ongoing flow of data helps to maintain and ensure the consistency and reliability of the system as a whole.

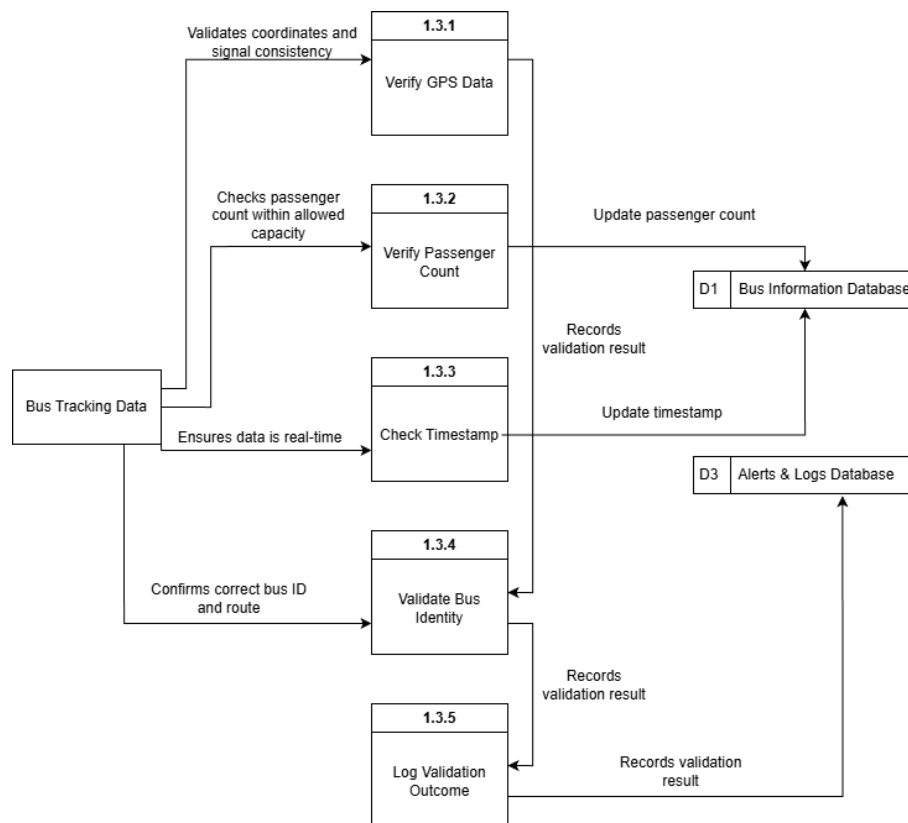


Figure 3.7: Data Flow Diagram Level 3 Bus Tracking Update Flow (DFD3)

3.15 Data Flow Diagram Level 3 Student Alert and Driver Response Flow (DFD3)

The logic behind alert handling and the driver response logic as represented in DFD Level 3 Whenever a student reports an alert, the system checks whether the alert conditions are met and logs the alert into the Alerts & Logs Store. Notification of relevant drivers according to the rules defined in the system. Drivers are then able to update the shuttle status, and it is recorded and displayed through the system. In this flow, you can see how events originating from the end-user get processed and how the system responds with actions to resolve an operational issue quickly and efficiently.

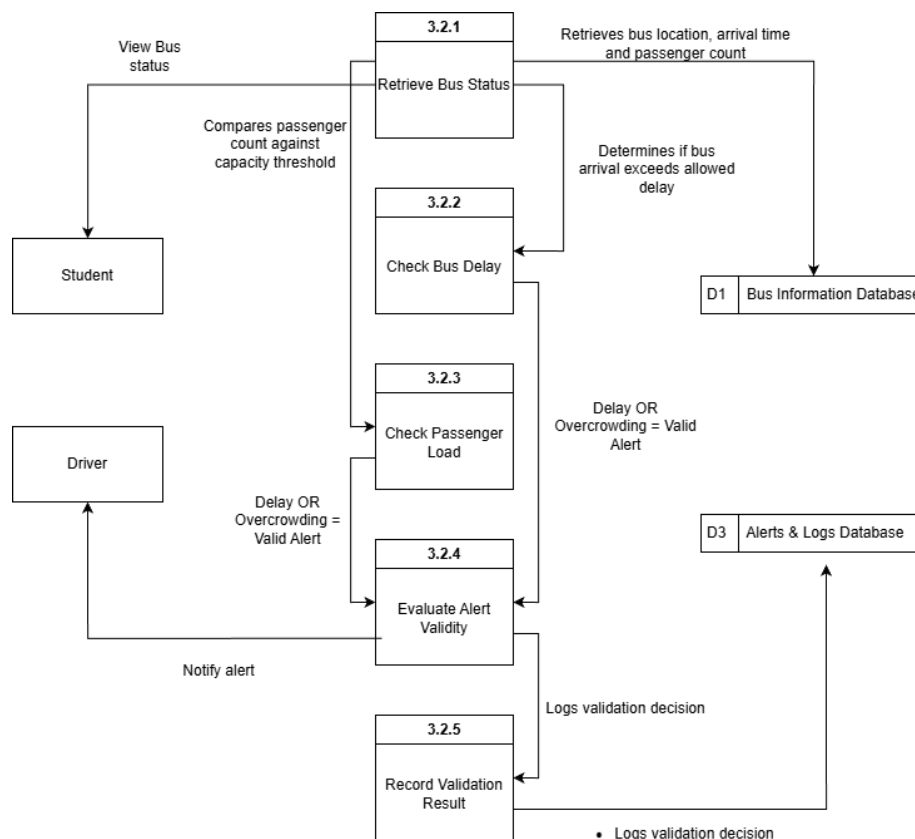


Figure 3.8: Data Flow Diagram 3 Student Alert and Driver Response (DFD3)

3.16 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram describes the database structure of Go-Campus system, identifying entities and relationships between them. Core entities are Student, Driver, Admin, Bus, Alert, and BusTracking. The relationships so students who trigger alerts, drivers who respond to alerts, and buses that produce tracking data are all well defined. It enforces data normalization, reduces redundancy, and allows for easy access of data. This type of database is a crucial piece of technology to keep real-time performance and system scalability.

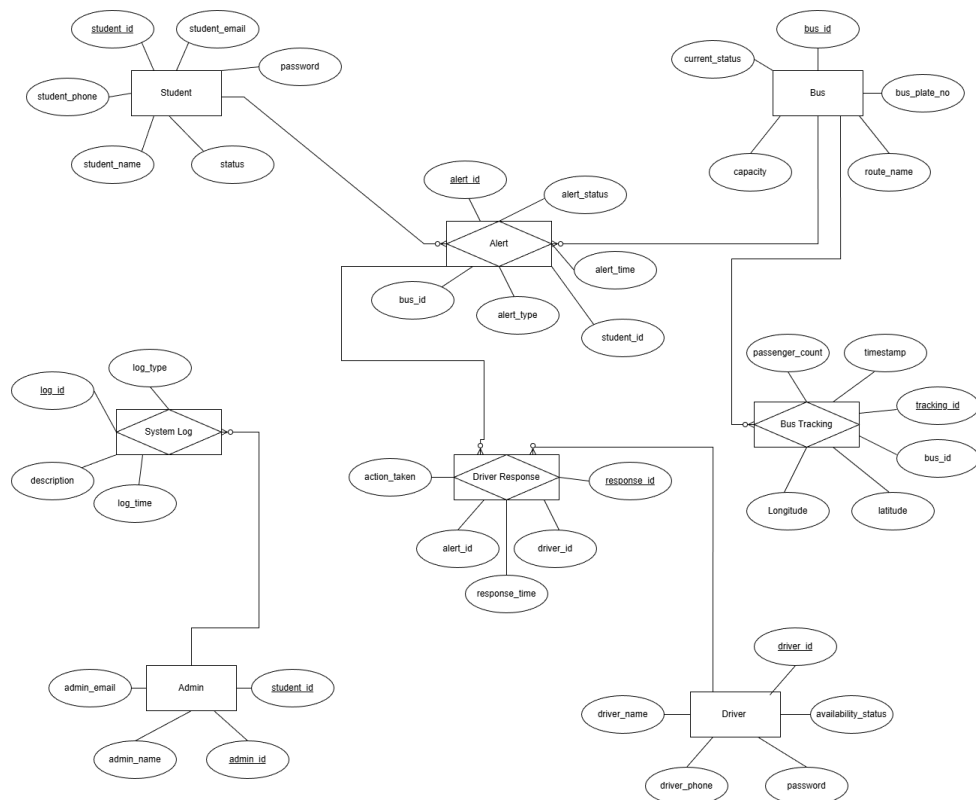


Figure 3.9 : Entity Relationship Diagram (ERD)

3.17 Flowchart IOT Data Collection and Update Process

IoT Data Collection and storage flow. Flowchart 1 Sensors collect the GPS and passenger count data and transmit data to the system. The Bus Information Store validates these data as inputs before storing them. This flowchart simplifies the underlying logic of how the operation works and identifies critical points in the decision-making process in the system.

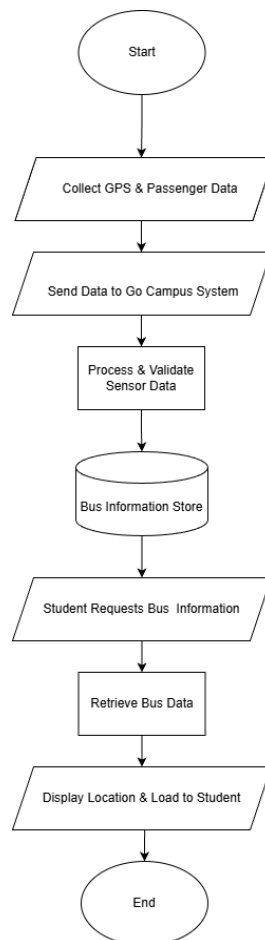


Figure 3.10: Flowchart IOT Data Collection and Update Process

3.18 Flowchart Student Alert and Driver Response

The alert submission and resolution process is depicted step by step in Flowchart 2. The process begins when a student notices a problem and raises an alert via the mobile app. The system then processes and logs the alert, measures if the driver has been notified, and tracks the driver response. Shuttle status is then updated throughout the entire system. This flowchart shows how user actions trigger system events, and how those events are resolved by synchronized system behaviors.

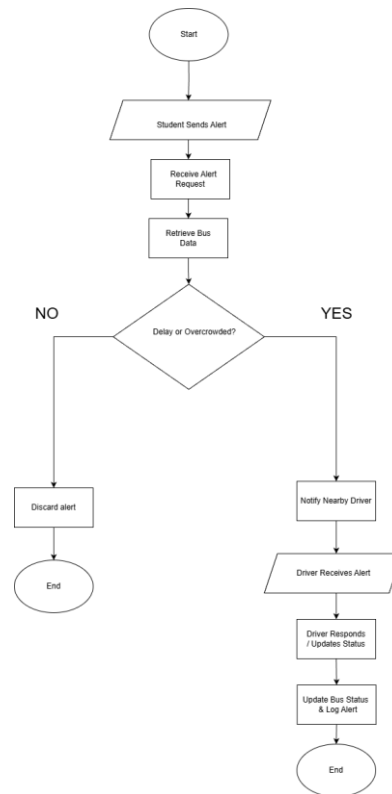


Figure 3.11: Flowchart Student Alert and Driver Response

3.19 Proof of Concept

In making an effort to show the practicality and usability of the solution, a proof of concept (PoC) was created for the Go-Campus Shuttle Management System. This prototype combines IoT based shuttle tracking hardware setup and a mobile app interface to test the real time monitoring and alert system. Live GPS location updates and passenger count readings from the ESP32 microcontroller and sensors were represented with simulated and dummy data. The system effectively provided students with shuttle location and occupancy information, and the event-based rule rule-driven naturally increased capacity to alert triggers automatically, when appropriate, during dynamic condition deviations (limit, e.g., delay and overcrowding). Also, drivers mean received notifications and updated shuttle status, while administrators were able to supervise system activity via its management features. The Go-Campus framework is practical and has the potential for use in smart campus transportation due to its real-time tracking and proven automated event handling and device-to-user communication, as shown by this proof of concept.

3.19.1 Login Page for Users

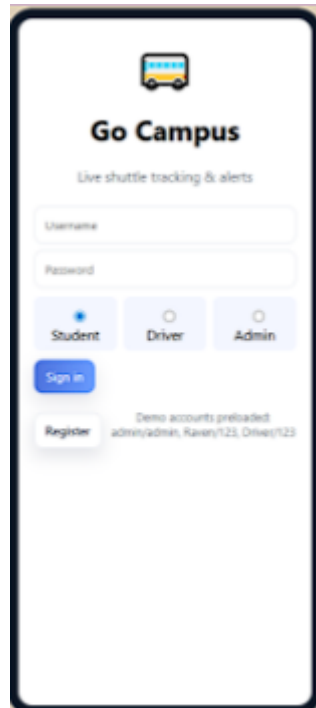


Figure 3.12: Login Page for Users

Go-Campus mobile application has the login and registration module that is secure and can support 3 major roles: the students, drivers and administrator. This page is where the user enters the system, and only users who have access to the platform can enter according to their assigned role. Register new users the process is giving the system some basic info about the new account, and it'll get stored in the system database for future authenticity. Students are routed to a shuttle tracking portal upon logging in, drivers gain access to alert notifications and status update functions, and administrative users receive top-level system control for managing accounts and monitoring operations. Implementing this role-based system login increases system security, allows for greater ease of use and guarantees that only access is granted to data necessary for each users role within the Go-Campus shuttle management system.

3.19.2 Student Dashboard



Figure 3.13: Student Interface



Figure 3.14: Passenger Count Function

The Student Dashboard is the home screen for students on the Go-Campus mobile app and contains the in-depth and real-time shuttle tracking functionalities. The dashboard aims to enhance the efficiency and comfort efficiency of the campus transport by showing the real-time updates regarding the location of the shuttles, which are acquired through GPS tracking from the IoT hardware module mounted in every bus. The map interface allows students to get an idea of where the shuttles are in proximity to each other, providing students the ability to estimate shuttle arrival times guaranteeing more efficient travel planning. The dashboard also has a passenger count feature, which enables students who plan to ride the shuttle to see how full the shuttle is at the time, addressing the potential impacts of overcrowding on comfort and safety. There is also a “Delay Reporting” or “Alert” button that enables students to promptly inform the system of missed shuttles or service disruptions. The alert mechanism powers the event-driven algorithm rule which assess the reported states and exposes an alert to the driver if required. In addition, the dashboard includes checkpoint and route progress details, illustrating which stops have been reached, where the shuttle is currently located, and where it will be heading next, helping students better visualize the movement of shuttles across campus. Through the Student Dashboard the user experience is greatly enhanced, there is less ambiguity regarding wait times, and data that enable smarter, more responsive operation of shuttles help the university.

3.19.3 Driver Dashboard

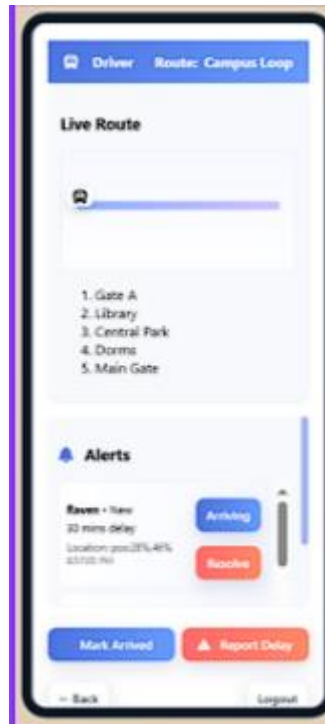


Figure 3.15: Driver Dashboard

To assist shuttle drivers in managing real-time operations and effectively respond to student transportation needs, a Driver Dashboard has been developed in the Go-Campus mobile application. This interface offers drivers live route guidance that allows them to reference a current position of the shuttle as well as the next checkpoints and scheduled stops on campus. The dashboard helps drivers maintain situational awareness of their schedules, coordinate their service better, and show real-time progress on a route. Furthermore, embedded within the Driver Dashboard is an alert notification window which reports for things like reported shuttle delays/overcrowded situations, filed by students. Alerts are linked with vital data, such as location of the student and the shuttle route it affects, so drivers can respond quickly. Drivers have control buttons Within the

shuttle system to mark arrived, delayed and resolve alert to do operational push updates. The database stores these updates and shows immediately on the student interface, so there is transparency, and little uncertainty for passengers. As a whole, the Driver Dashboard increases driver responsiveness, increases shuttle service reliability, and strengthens real-time communication between students and campus transportation providers

3.19.4 Admin Dashboard

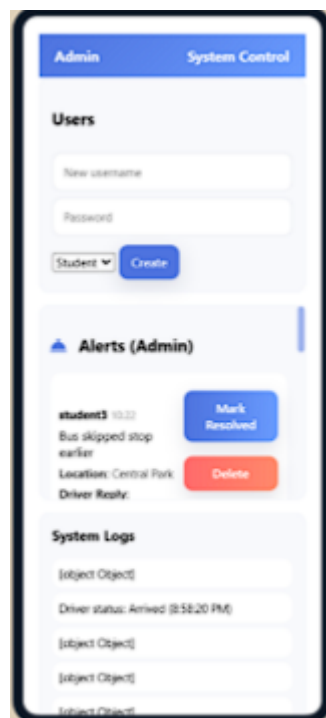


Figure 3.16: Admin Dashboard

Admin Dashboard is the Heart Of Go-Campus Shuttle Management System For Admin Where they Can Manage Whole System. A dashboard that serves as the core management console, from where administrators can manage user account by registering, updating, maintaining student/driver profile etc. With role-based access

control, the admin ensures that every user can only access the relevant permissions in the system. As well as, it allows administrators to check the system operational reports, alerts generated by the students due to missed shuttles and driver responses to help them track the performance of the service over time and also able to identify patterns of repeated problems e.g. message about delay or overcrowding. Activity logs: The system keeps a log of every important aspect in a shuttleroad, such as logins history, where alerts had been sent and shuttle status. These logs enable transparency, accountability, and analysis for later efforts to improve the system. In brief, Admin dashboard enables smooth operation of the system, effective management of users, and constant monitoring of campus transport operations.

3.20 Summary of Methodology

In this chapter, the system design and methodology of the Go-Campus Shuttle Management System are presented. The overarching framework was presented to show the combination IoT hardware with mobile application to deliver live shuttle tracking and passenger observation. Context diagram System flow diagram Data flow diagram (Level-0, Level-1) Logic flow Design entity relationship diagram The static and dynamic aspects of the system can be expressively illustrated using a set of system diagrams. Together these diagrams overview how data is collected, processed, stored and ultimately served to different users of the system.

Also described was the Agile development methodology and rationale for using it as our approach for this project. Agile showed the iterative development of the system developing in iterative stages which provided continuous improvements, early testing

and changes in hardware and software can be integrated into the system. It also define the system requirement, include the functional and non function requirement of the Go-Campus system, to provide the expectation of the user and the constraint to run the system.

In summary, this chapter is pivotal for the implementation phase of the Go-Campus project, as it provides a robust technical foundation by detailing the system architecture and development methodology as well as the operational needs to support the successful Evolvment of Go-Campus Shuttle Management System.

REFERENCES

- Subramaniam, P., Shanmugam, U., Balasubramanian, K., & Manikandan, M. (2019). Passenger and luggage weight monitoring system for public transport. *International Journal of Advanced Computer Science and Applications*, 10(6), 528–534. Retrieved from https://thesai.org/Downloads/Volume10No6/Paper_62-Passenger_and_Luggage_Weight_Monitoring_System.pdf
- Sujitha, K., Karthika, R., & Sharmila, D. (2020). IoT based bus arrival time prediction using artificial neural network for smart public transport system (SPTS). *International Journal of Nonlinear Analysis and Applications*, 11(1), 403–412. Retrieved from <https://inass.org/2020/2020022929.pdf>
- Karthik, S., Prakash, V., Srikanth, A., & Rohan, M. (2023). Bus tracking and arrival prediction system. *International Journal of Innovative Science and Research Technology*, 8(11), 2426–2432. Retrieved from <https://www.ijisrt.com/assets/upload/files/IJISRT23NOV2445.pdf>
- Mohamed, A., Yusof, N., Rahman, F., Hamzah, N., & Ishak, M. (2023). Passenger occupancy estimation in vehicles: A review of methods. *Sustainability*, 15(2), 1332. Retrieved from <https://www.mdpi.com/2071-1050/15/2/1332>
- Zhang, J., Guo, R., & Li, W. (2024). Research on dynamic scheduling and route optimization strategy of flex-route transit considering travel choice preference of passengers. *Systems*, 12(4), 138. Retrieved from <https://www.mdpi.com/2079-8954/12/4/138>
- Lin, F., & Hsieh, H.-P. (2022). Multicriteria route planning for in-operation mass transit under urban data. *Applied Sciences*, 12(6), 3127. Retrieved from <https://www.mdpi.com/2076-3417/12/6/3127>
- Zhou, X., Liu, Q., & Wang, R. (2024). Transit dynamic operation optimization using combination of stop-skipping control and local route optimization. *Applied Sciences*, 14(17), 7783. Retrieved from <https://www.mdpi.com/2076-3417/14/17/7783>
- Aydin, M. M., Sokolovskij, E., Jaskowski, P., & Matijošius, J. (2025). Service management of employee shuttle service under inhomogeneous fleet constraints using dynamic linear programming: A case study. *Applied Sciences*, 15(9), 4604. Retrieved from <https://www.mdpi.com/2076-3417/15/9/4604>
- Wei, M., Yang, C., & Liu, T. (2022). An integrated multi-objective optimization for dynamic airport shuttle bus location, route design and departure frequency setting problem. *International Journal of Environmental Research and Public Health*, 19(21), 14469. Retrieved from <https://www.mdpi.com/1660-4601/19/21/14469>
- Sun, Q., Hu, D. W., Chien, S., Jiang, J., Gao, T. Y., & Jiang, R. (2022). Dynamic bus routing optimization for demand-responsive feeder transit considering stochastic bus arrival time. *Faculty Publications (NJIT)*, 2578. Retrieved from https://digitalcommons.njit.edu/fac_pubs/2578/